PROBLEM STATEMENT 1

The Evidence Protector – Automated Log Integrity Monitor

Problem

Modern systems generate millions of log lines, making manual inspection of logs impractical. When hackers breach systems, they often delete log entries, creating suspicious time gaps that go undetected without proper tooling.

Challenge

Build a Python-based CLI tool that scans large log files, extracts timestamps, and automatically flags suspicious time gaps that may indicate log tampering.

What Students Can Do:

- **Gap Detection:** Parse logs line-by-line and identify time gaps beyond a configurable threshold
- **Forensic Reporting:** Output clear, actionable reports showing gap start, end, and duration
- **Error Handling:** Gracefully handle malformed lines without crashing

Starter Tech Stack:

- **Language:** Python (standard libraries only)
- **Input:** .log files | Output: Terminal report / CSV / JSON

PROBLEM STATEMENT 2

Smart Knowledge Navigator – Agentic RAG for Distributed Content

Problem

Team knowledge is scattered across Confluence pages, PDFs, and shared documents. Users waste significant time manually browsing multiple sources just to find a simple answer.

Challenge

Build a multi-agent RAG (Retrieval-Augmented Generation) system that understands a user's question, searches across multiple documents, and returns a cited, well-supported answer.

What Students Can Do:

- **Agent Design:** Implement Planning, Retrieval, and Synthesis agents
- **Document Ingestion:** Build a pipeline to ingest PDFs, HTML pages, and Confluence exports
- **Citation Output:** Return answers with references to original source content

Starter Tech Stack:

- **Frameworks:** LangChain, LlamaIndex
- **LLMs:** Open-source models via HuggingFace
- **UI:** Streamlit or Gradio

PROBLEM STATEMENT 3

Viral Claim Radar – Fact-Check Copilot for Social Feeds

Problem

Misinformation spreads rapidly across Instagram, Twitter, and WhatsApp. Users have no quick, reliable way to verify the credibility of claims they encounter in their social feeds.

Challenge

Build a fact-checking tool that extracts claims from text or screenshot posts, runs semantic search over a verified offline corpus, and classifies each claim as Supported, Refuted, or Uncertain with evidence.

What Students Can Do:

- **Claim Extraction:** Use NLP to identify verifiable claims from raw social media text or screenshots
- **RAG Verification:** Search a curated corpus of fact-check portals and government advisories
- **Stance Classification:** Return confidence scores and supporting evidence cards

Starter Tech Stack:

- **NLP:** SpaCy, HuggingFace Transformers
- **RAG:** FAISS + LangChain
- **UI:** React or browser extension